

APOSTILA COMPLETA DE ESTUDO

JavaScript para Web, DOM, Eventos, Listas, Persistência, Web Moderna e Arrow Functions

Material de estudo baseado nos PDFs enviados na conversa.

Organizado para revisão, prática e preparação para atividades/provas.

Aluno: Danrley Correia Ramos

Data: 12/06/2026

Como estudar: Leia um capítulo por vez, copie os exemplos no VS Code, altere valores e tente resolver os exercícios sem olhar o gabarito. Depois, compare sua resposta com a explicação.

Sumário

1. Introdução à programação web com JavaScript
2. JavaScript na Web e introdução ao DOM
3. Manipulando o DOM
4. Construção dinâmica de interfaces
5. Eventos web e experiência do usuário
6. Listas, arrays e estruturas de dados no front-end
7. Manipulação avançada de eventos e persistência de dados
8. Desenvolvimento web moderno: HTTP, APIs, JSON, Fetch, WebSocket e segurança
9. Arrow Functions no JavaScript moderno
10. Projetos práticos integradores
11. Plano de estudo e revisão rápida
12. Exercícios finais com gabarito
13. Glossário e referências dos materiais enviados

1. Introdução à programação web com JavaScript

Objetivos do capítulo

- Entender o papel do JavaScript no ecossistema web.
- Diferenciar HTML, CSS e JavaScript.
- Conhecer variáveis, tipos de dados, operadores, condicionais e laços.
- Construir a base lógica necessária para manipular páginas web.

JavaScript é a linguagem que permite dar comportamento às páginas web. Uma página pode ser bonita usando HTML e CSS, mas sem JavaScript ela tende a ser estática. Com JavaScript, o usuário pode clicar, digitar, filtrar dados, salvar informações, buscar dados de uma API e ver a tela mudar sem recarregar tudo.

Tecnologia	Função	Exemplo simples
HTML	Estrutura da página	Título, parágrafo, botão, formulário
CSS	Aparência visual	Cores, fontes, espaçamentos, layout
JavaScript	Comportamento/interatividade	Clique, validação, cálculo, busca de dados

Analogia: HTML é o esqueleto da casa, CSS é a decoração e JavaScript é a parte elétrica e funcional: acende a luz, abre o portão e responde às ações do usuário.

1.1 Inserindo JavaScript no HTML

Existem três formas comuns de usar JavaScript em uma página: diretamente no atributo do elemento, dentro da tag `<script>` ou em um arquivo externo `.js`. A forma mais organizada é usar arquivo externo.

```

<!-- Opção 1: script dentro do HTML -->
<button onclick="alert('Olá!')">Clique</button>

<!-- Opção 2: script interno -->
<script>
  console.log('Página carregada');
</script>

<!-- Opção 3: arquivo externo, mais profissional -->
<script src="script.js"></script>

```

Boa prática: Prefira colocar o JavaScript em um arquivo separado, como `script.js`. Isso deixa o HTML mais limpo e facilita manutenção.

1.2 Variáveis: let e const

Variável é um espaço com nome usado para guardar informação. Use `let` quando o valor pode mudar. Use `const` quando o valor não deve ser reatribuído.

```

let idade = 30;
idade = 31; // permitido

const nome = 'Danrley';
// nome = 'João'; // erro, pois const não deve mudar

```

Comando	Quando usar	Exemplo
let	Quando o valor pode mudar	let contador = 0
const	Quando o valor deve permanecer fixo	const PI = 3.14
var	Sintaxe antiga; evite em código moderno	var nome = "Ana"

1.3 Tipos de dados

Tipo de dado é a natureza da informação guardada. O JavaScript trabalha com textos, números, booleanos, objetos, arrays e valores especiais como null e undefined.

```
const nome = 'Maria';      // string
const idade = 22;          // number
const aprovado = true;     // boolean
const endereco = null;    // vazio intencional
let telefone;              // undefined

const notas = [8, 7, 10];  // array
const aluno = {            // object
  nome: 'Maria',
  idade: 22
};
```

Tipo	Significado	Exemplo
string	Texto	"Olá"
number	Número inteiro ou decimal	10, 7.5
boolean	Verdadeiro ou falso	true, false
array	Lista de valores	[1, 2, 3]
object	Conjunto de propriedades	{ nome: "Ana" }
null	Ausência intencional de valor	let x = null
undefined	Valor ainda não definido	let x;

1.4 Operadores

Operadores permitem calcular, comparar e combinar informações.

```
const soma = 10 + 5;      // 15
const subtracao = 10 - 5; // 5
const produto = 10 * 5;   // 50
const divisao = 10 / 5;   // 2
const resto = 10 % 3;     // 1

console.log(10 > 5);      // true
console.log(10 === '10'); // false, compara valor e tipo
console.log(10 == '10');  // true, evite usar ==
```

Regra prática: Use === em vez de ==. O operador === compara valor e tipo, evitando resultados inesperados.

1.5 Condicionais

Condicional é uma decisão. O código faz uma coisa se a condição for verdadeira e outra se for falsa.

```
const nota = 7;

if (nota >= 6) {
  console.log('Aprovado');
} else {
  console.log('Reprovado');
}
```

Também é possível testar várias possibilidades com else if:

```
const media = 8.5;

if (media >= 9) {
  console.log('Excelente');
} else if (media >= 7) {
  console.log('Bom');
} else if (media >= 6) {
  console.log('Regular');
} else {
```

```
console.log('Reprovado');
}
```

1.6 Laços de repetição

Laços repetem comandos. São úteis quando você precisa percorrer listas ou executar algo várias vezes.

```
for (let i = 1; i <= 5; i++) {
  console.log('Número ' + i);
}

let contador = 1;
while (contador <= 3) {
  console.log('Contador: ' + contador);
  contador++;
}
```

Cuidado: Todo laço precisa ter uma condição de parada. Se a condição nunca ficar falsa, o programa entra em loop infinito.

1.7 Funções

Função é um bloco de código reutilizável. Você cria uma vez e chama sempre que precisar.

```
function saudacao(nome) {
  return 'Olá, ' + nome + '!';
}

const mensagem = saudacao('Danrley');
console.log(mensagem);
```

Funções ajudam a organizar o código, evitar repetição e deixar a lógica mais clara.

Exercícios para fixação

1. Crie uma variável nome e mostre uma mensagem de boas-vindas no console.
2. Crie uma condição que diga se uma pessoa é maior ou menor de idade.
3. Crie um for que mostre os números de 1 até 10.
4. Crie uma função chamada calcularDobro que receba um número e retorne o dobro.

Gabarito comentado

1. Use `const nome = "Danrley"; console.log("Bem-vindo, " + nome);`
2. Use `if (idade >= 18)` para maior de idade e `else` para menor.
3. Use `for (let i = 1; i <= 10; i++) { console.log(i); }`.
4. `function calcularDobro(numero) { return numero * 2; }`

2. JavaScript na Web e introdução ao DOM

Objetivos do capítulo

- Entender o que é DOM e por que ele é importante.
- Selecionar elementos HTML usando JavaScript.
- Alterar conteúdo, atributos e estilos básicos.
- Compreender a relação entre árvore DOM e elementos da página.

DOM significa Document Object Model. Quando o navegador carrega uma página HTML, ele transforma o documento em uma estrutura de objetos. Essa estrutura funciona como uma árvore. Cada tag vira um nó que pode ser acessado e alterado pelo JavaScript.



Ideia principal: Você não altera o arquivo HTML original diretamente. Você altera a representação da página na memória do navegador, ou seja, o DOM.

2.1 Selecionando elementos

Para manipular algo na página, primeiro você precisa selecionar esse elemento.

```

<h1 id="titulo">Minha página</h1>
<p class="descricao">Texto inicial</p>
<button id="botao">Clique</button>

const titulo = document.getElementById('titulo');
const descricao = document.querySelector('.descricao');
const botao = document.querySelector('#botao');

```

Método	O que seleciona	Exemplo
getElementById	Elemento pelo id	document.getElementById("titulo")
querySelector	Primeiro elemento que combina com o seletor	document.querySelector(".card")
querySelectorAll	Todos os elementos que combinam	document.querySelectorAll("li")
getElementsByClassName	Elementos por classe	document.getElementsByClassName("item")
getElementsByTagName	Elementos pela tag	document.getElementsByTagName("p")

2.2 Alterando textos

Depois de selecionar, você pode alterar o texto com `textContent`.

```

const titulo = document.getElementById('titulo');
titulo.textContent = 'Título alterado pelo JavaScript';

```

2.3 Alterando atributos

Atributos são informações dentro da tag HTML, como `src`, `href`, `alt`, `id` e `class`.

```



const foto = document.getElementById('foto');
foto.setAttribute('src', 'foto2.png');
foto.setAttribute('alt', 'Nova foto');

```

2.4 Alterando estilos

A propriedade `style` permite mudar CSS diretamente pelo JavaScript.

```

const titulo = document.getElementById('titulo');
titulo.style.color = 'blue';
titulo.style.fontSize = '32px';
titulo.style.backgroundColor = 'yellow';

```

Atenção ao nome da propriedade: No CSS usamos `background-color`. No JavaScript usamos `backgroundColor`, sem hífen e com letra maiúscula na segunda palavra.

2.5 Exercício guiado: contador simples

```
<h1 id="contador">0</h1>
<button id="aumentar">Aumentar</button>

<script>
  const contador = document.getElementById('contador');
  const botao = document.getElementById('aumentar');
  let numero = 0;

  botao.addEventListener('click', function () {
    numero++;
    contador.textContent = numero;
  });
</script>
```

Esse exemplo já junta DOM e evento. O botão é selecionado, o clique é escutado e o texto do h1 é atualizado.

Exercícios para fixação

1. Selecione um h1 pelo id e altere seu texto.
2. Crie um botão que muda a cor de um parágrafo.
3. Crie uma imagem e altere o atributo src pelo JavaScript.
4. Explique com suas palavras o que é DOM.

Gabarito comentado

1. Use `document.getElementById("idDoH1").textContent = "Novo texto"`.
2. Selecione o parágrafo e no clique faça `paragrafo.style.color = "red"`.
3. Use `imagem.setAttribute("src", "nova-imagem.png")`.
4. DOM é a representação da página HTML em forma de objetos que o JavaScript consegue acessar e modificar.

3. Manipulando o DOM

Objetivos do capítulo

- Diferenciar `textContent`, `innerText` e `innerHTML`.
- Alterar classes CSS com `classList`.
- Ler valores de campos de formulário.
- Evitar erros comuns ao manipular elementos.

3.1 `textContent`, `innerText` e `innerHTML`

Essas propriedades alteram ou leem conteúdo, mas não são iguais.

Propriedade	O que faz	Quando usar
<code>textContent</code>	Lê ou altera texto puro	Quando quer segurança e simplicidade
<code>innerText</code>	Considera texto visível e estilo aplicado	Quando precisa do texto como aparece na tela
<code>innerHTML</code>	Lê ou insere HTML	Quando precisa criar tags dentro do elemento

```
const caixa = document.getElementById('caixa');

caixa.textContent = '<strong>Olá</strong>'; // aparece como texto
caixa.innerHTML = '<strong>Olá</strong>'; // aparece em negrito
```

Segurança: Evite colocar conteúdo digitado pelo usuário diretamente em innerHTML. Isso pode abrir espaço para ataques de injeção de código. Para texto do usuário, prefira `textContent`.

3.2 Manipulando classes com `classList`

Em vez de alterar vários estilos individualmente, normalmente é melhor criar classes no CSS e ativar/desativar essas classes no JavaScript.

```
.destaque {
  background-color: yellow;
  font-weight: bold;
}

.erro {
  color: red;
}

const mensagem = document.getElementById('mensagem');

mensagem.classList.add('destaque');
mensagem.classList.remove('erro');
mensagem.classList.toggle('ativo');
console.log(mensagem.classList.contains('destaque'));
```

Método	Função
<code>add</code>	Adiciona uma classe
<code>remove</code>	Remove uma classe
<code>toggle</code>	Alterna: se tem remove, se não tem adiciona
<code>contains</code>	Verifica se a classe existe

3.3 Lendo valores de inputs

Campos de formulário usam a propriedade `value`. O texto digitado pelo usuário fica em `input.value`, `textarea.value` ou `select.value`.

```
<input id="nome" placeholder="Digite seu nome">
<button id="enviar">Enviar</button>
<p id="resultado"></p>

const nome = document.getElementById('nome');
const enviar = document.getElementById('enviar');
const resultado = document.getElementById('resultado');

enviar.addEventListener('click', function () {
  resultado.textContent = 'Olá, ' + nome.value;
});
```

3.4 Erros comuns

Erro	Causa provável	Como corrigir
Cannot read properties of null	O seletor não encontrou o elemento	Confira id/classe ou coloque o script no final do body
Texto não muda	Variável aponta para elemento errado	Teste <code>console.log(elemento)</code>
Classe não aplica	Nome da classe diferente do CSS	Verifique grafia e maiúsculas/minúsculas
Valor do input vem vazio	Está lendo antes do clique/digitação	Leia <code>input.value</code> dentro do evento

Exercícios para fixação

1. Crie um input e mostre o que foi digitado em um parágrafo.

2. Crie um botão que alterna a classe escuro no body.
3. Explique a diferença entre `textContent` e `innerHTML`.
4. Crie uma validação simples: se o input estiver vazio, mostre mensagem de erro.

Gabarito comentado

1. Use `input.value` dentro de um evento de click ou input.
2. `document.body.classList.toggle("escuro")` dentro do clique.
3. `textContent` insere texto puro; `innerHTML` interpreta tags HTML.
4. `if (input.value.trim() === "") { mensagem.textContent = "Preencha o campo"; }`

4. Construção dinâmica de interfaces

Objetivos do capítulo

- Criar elementos HTML com JavaScript.
- Inserir elementos na página usando `appendChild` e `append`.
- Remover elementos com `remove` e `removeChild`.
- Construir pequenas interfaces dinâmicas, como cards e listas.

Construção dinâmica significa criar elementos pela lógica do JavaScript em vez de escrever tudo manualmente no HTML. Isso é essencial quando a quantidade de itens depende de dados: produtos, tarefas, mensagens, usuários, comentários etc.

4.1 createElement

```
const paragrafo = document.createElement('p');
paragrafo.textContent = 'Texto criado pelo JavaScript';

document.body.appendChild(paragrafo);
```

O elemento nasce na memória. Ele só aparece na tela depois que é inserido em algum lugar do DOM.

4.2 appendChild e append

Método	Função	Observação
<code>appendChild</code>	Inserir um nó filho	Aceita um nó por vez
<code>append</code>	Inserir nós ou textos	Mais flexível e moderno
<code>prepend</code>	Inserir no começo	Útil para novos itens aparecerem primeiro

```
const lista = document.getElementById('lista');
const item = document.createElement('li');
item.textContent = 'Estudar DOM';

lista.appendChild(item);
```

4.3 Criando cards de produto

```
<div id="produtos"></div>

const area = document.getElementById('produtos');

const card = document.createElement('article');
card.classList.add('card');

const titulo = document.createElement('h2');
```

```

titulo.textContent = 'Notebook';

const preco = document.createElement('p');
preco.textContent = 'R$ 3.000,00';

const botao = document.createElement('button');
botao.textContent = 'Comprar';

card.append(titulo, preco, botao);
area.appendChild(card);

```

4.4 Removendo elementos

```

const item = document.querySelector('li');
item.remove();

```

Também existe `removeChild`, que remove um filho a partir do elemento pai:

```

const lista = document.getElementById('lista');
const primeiroItem = lista.querySelector('li');

lista.removeChild(primeiroItem);

```

4.5 Exercício guiado: lista de tarefas

```

<input id="tarefa" placeholder="Digite uma tarefa">
<button id="adicionar">Adicionar</button>
<ul id="lista"></ul>

const input = document.getElementById('tarefa');
const adicionar = document.getElementById('adicionar');
const lista = document.getElementById('lista');

adicionar.addEventListener('click', function () {
  const texto = input.value.trim();

  if (texto === '') {
    alert('Digite uma tarefa');
    return;
  }

  const item = document.createElement('li');
  item.textContent = texto;

  item.addEventListener('click', function () {
    item.remove();
  });

  lista.appendChild(item);
  input.value = '';
});

```

Exercícios para fixação

1. Crie uma div pelo JavaScript e coloque texto dentro dela.
2. Crie uma lista onde cada clique no botão adiciona um novo item.
3. Faça cada item criado poder ser removido ao receber clique.
4. Crie um card com título, descrição e botão usando `createElement`.

Gabarito comentado

1. Use `document.createElement("div")`, `textContent` e `appendChild`.
2. No clique, crie um `li` e adicione na `ul`.
3. Adicione `item.addEventListener("click", () => item.remove())`.
4. Crie `article`, `h2`, `p` e `button`, depois use `append` para montar o card.

5. Eventos web e experiência do usuário

Objetivos do capítulo

- Entender o que são eventos no navegador.
- Usar `addEventListener` como padrão profissional.
- Relacionar ações do usuário com respostas do sistema.
- Validar formulários e melhorar a experiência do usuário.

Evento é qualquer ação ou acontecimento que pode ser percebido pelo navegador: clique, digitação, envio de formulário, movimento do mouse, carregamento da página e muito mais. JavaScript permite escutar esses eventos e executar funções em resposta.

Evento	Quando acontece	Exemplo de uso
click	Usuário clica	Abrir menu, adicionar item
input	Usuário digita/altera campo	Busca em tempo real
submit	Formulário é enviado	Validar dados antes de enviar
mouseover	Mouse passa por cima	Destacar elemento
keydown	Tecla é pressionada	Atalhos de teclado
DOMContentLoaded	HTML terminou de carregar	Iniciar scripts com segurança

5.1 `addEventListener`

```
const botao = document.getElementById('botao');

botao.addEventListener('click', function () {
  console.log('Botão clicado');
});
```

A estrutura é: `elemento.addEventListener(tipoDoEvento, funçãoQueExecuta)`.

5.2 Objeto event

A função do evento pode receber um parâmetro que representa informações sobre o evento. Normalmente chamamos esse parâmetro de `event` ou `e`.

```
const input = document.getElementById('nome');

input.addEventListener('input', function (event) {
  console.log('Valor digitado:', event.target.value);
});
```

`event.target` é o elemento que disparou o evento.

5.3 Validação de formulário

```
<form id="formulario">
  <input id="email" placeholder="Digite seu email">
  <button type="submit">Cadastrar</button>
</form>
<p id="mensagem"></p>

const formulario = document.getElementById('formulario');
const email = document.getElementById('email');
const mensagem = document.getElementById('mensagem');

formulario.addEventListener('submit', function (event) {
  event.preventDefault();

  if (email.value.trim() === '') {
    mensagem.textContent = 'Preencha o email';
    mensagem.classList.add('erro');
    return;
  }
});
```

```

}

mensagem.textContent = 'Cadastro realizado com sucesso';
mensagem.classList.remove('erro');
});

```

preventDefault: Em formulários, preventDefault impede que a página recarregue automaticamente. Assim, você consegue validar e controlar o envio com JavaScript.

5.4 Experiência do usuário

Eventos devem melhorar a experiência. Um bom sistema dá retorno claro ao usuário: mensagens de erro, confirmação, campos obrigatórios, loading, estados visuais e feedback imediato.

Situação	Resposta ruim	Resposta melhor
Campo vazio	Nada acontece	Mostrar mensagem clara
Processando dados	Tela parada	Mostrar carregando...
Ação concluída	Usuário não sabe se salvou	Mostrar sucesso
Erro de validação	Erro genérico	Dizer qual campo precisa corrigir

Exercícios para fixação

1. Crie um botão que mostra um alert ao ser clicado.
2. Crie um input que atualiza um parágrafo enquanto o usuário digita.
3. Crie um formulário que não permite envio com campo vazio.
4. Explique a função do preventDefault em formulários.

Gabarito comentado

1. Use `botao.addEventListener("click", () => alert("Olá"))`.
2. Use evento `input` e atualize `p.textContent` com `input.value`.
3. No `submit`, use `preventDefault` e `if (campo.value.trim() === '')`.
4. Ele cancela o comportamento padrão do navegador, como recarregar a página no `submit`.

6. Listas, arrays e estruturas de dados no front-end

Objetivos do capítulo

- Compreender arrays como listas de dados.
- Manipular arrays com `push`, `pop`, `shift`, `unshift`, `splice` e `indexOf`.
- Percorrer arrays com `for`, `forEach`, `map`, `filter` e `reduce`.
- Aplicar arrays em interfaces web reais.

Array é uma lista ordenada de valores. Em interfaces web, arrays aparecem o tempo todo: lista de produtos, tarefas, usuários, mensagens, notas, comentários e itens de carrinho.

```

const frutas = ['banana', 'maçã', 'uva'];

console.log(frutas[0]); // banana
console.log(frutas[1]); // maçã
console.log(frutas.length); // 3

```

Índice começa em zero: O primeiro item de um array está na posição 0, não na posição 1.

6.1 Métodos básicos

Método	O que faz	Exemplo
push	Adiciona no final	lista.push("novo")
pop	Remove do final	lista.pop()
unshift	Adiciona no começo	lista.unshift("novo")
shift	Remove do começo	lista.shift()
splice	Remove ou insere em posição específica	lista.splice(1, 1)
includes	Verifica se contém valor	lista.includes("Ana")
indexOf	Retorna índice do valor	lista.indexOf("Ana")

```
const tarefas = ['estudar', 'trabalhar'];

tarefas.push('revisar');
tarefas.unshift('acordar cedo');
tarefas.pop();

console.log(tarefas);
```

6.2 Percorrendo arrays

```
const nomes = ['Ana', 'Bruno', 'Carlos'];

for (let i = 0; i < nomes.length; i++) {
  console.log(nomes[i]);
}

nomes.forEach(function (nome) {
  console.log(nome);
});
```

6.3 map, filter e reduce

Esses métodos são muito usados no JavaScript moderno.

Método	Ideia principal	Retorno
map	Transforma cada item	Novo array do mesmo tamanho
filter	Filtra itens por condição	Novo array com parte dos itens
reduce	Acumula valores	Um único resultado

```
const numeros = [1, 2, 3, 4];

const dobrados = numeros.map(function (n) {
  return n * 2;
});

const pares = numeros.filter(function (n) {
  return n % 2 === 0;
});

const soma = numeros.reduce(function (total, n) {
  return total + n;
}, 0);

console.log(dobrados); // [2, 4, 6, 8]
console.log(pares);    // [2, 4]
console.log(soma);     // 10
```

6.4 Arrays de objetos

Em sistemas reais, normalmente cada item é um objeto com várias informações.

```
const produtos = [
  { nome: 'Notebook', preco: 3000, categoria: 'informática' },
  { nome: 'Mouse', preco: 50, categoria: 'informática' },
  { nome: 'Cadeira', preco: 800, categoria: 'móveis' }
];

const caros = produtos.filter(function (produto) {
  return produto.preco > 500;
});

const nomes = produtos.map(function (produto) {
  return produto.nome;
});

console.log(caros);
console.log(nomes);
```

6.5 Renderizando array na tela

```
<ul id="lista-produtos"></ul>

const lista = document.getElementById('lista-produtos');

produtos.forEach(function (produto) {
  const item = document.createElement('li');
  item.textContent = produto.nome + ' - R$ ' + produto.preco;
  lista.appendChild(item);
});
```

Exercícios para fixação

1. Crie um array com cinco nomes e mostre todos no console.
2. Use map para dobrar os números [2, 4, 6].
3. Use filter para pegar apenas produtos acima de R\$ 100.
4. Renderize um array de tarefas em uma ul no HTML.

Gabarito comentado

1. Use nomes.forEach(nome => console.log(nome)).
2. const dobrados = numeros.map(n => n * 2).
3. const caros = produtos.filter(p => p.preco > 100).
4. Percorra o array, crie li com createElement e adicione com appendChild.

7. Manipulação avançada de eventos e persistência de dados

Objetivos do capítulo

- Entender propagação de eventos: capturing e bubbling.
- Usar stopPropagation quando necessário.
- Implementar delegação de eventos.
- Guardar dados com localStorage e sessionStorage.
- Usar JSON.stringify e JSON.parse para salvar arrays e objetos.

7.1 Propagação de eventos

Quando você clica em um elemento dentro de outro, o evento pode passar por várias camadas. Isso se chama propagação. A fase mais comum é o bubbling: o evento começa no elemento clicado e sobe para os elementos pais.

```

<div id="caixa">
  <button id="botao">Clique</button>
</div>

const caixa = document.getElementById('caixa');
const botao = document.getElementById('botao');

caixa.addEventListener('click', function () {
  console.log('Clicou na caixa');
});

botao.addEventListener('click', function () {
  console.log('Clicou no botão');
});

```

Ao clicar no botão, pode aparecer primeiro "Clicou no botão" e depois "Clicou na caixa", porque o clique sobe para o elemento pai.

7.2 stopPropagation

```

botao.addEventListener('click', function (event) {
  event.stopPropagation();
  console.log('Clicou somente no botão');
});

```

Use com cuidado: stopPropagation pode resolver um problema pontual, mas também pode dificultar a manutenção se usado sem necessidade.

7.3 Delegação de eventos

Delegação é colocar o evento em um elemento pai e identificar qual filho foi clicado. Isso é ótimo para listas dinâmicas, porque itens criados depois continuam funcionando.

```

<ul id="lista">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>

const lista = document.getElementById('lista');

lista.addEventListener('click', function (event) {
  if (event.target.tagName === 'LI') {
    console.log('Clicou em:', event.target.textContent);
  }
});

```

Quando usar delegação: Use quando você tem muitos elementos semelhantes ou quando os elementos serão criados dinamicamente depois do carregamento da página.

7.4 localStorage e sessionStorage

Recurso	Duração	Exemplo de uso
localStorage	Permanece mesmo após fechar o navegador	Preferência de tema, tarefas salvas
sessionStorage	Dura apenas enquanto a aba/sessão existir	Dados temporários de navegação

```

localStorage.setItem('nome', 'Danrley');
const nome = localStorage.getItem('nome');
localStorage.removeItem('nome');
localStorage.clear();

```

7.5 Salvando arrays e objetos

O localStorage só guarda texto. Por isso, objetos e arrays precisam ser convertidos para texto com JSON.stringify. Para recuperar, usamos JSON.parse.

```
const usuario = {
  nome: 'Danrley',
  curso: 'JavaScript'
};

localStorage.setItem('usuario', JSON.stringify(usuario));

const salvo = localStorage.getItem('usuario');
const usuarioConvertido = JSON.parse(salvo);

console.log(usuarioConvertido.nome);
```

7.6 Projeto guiado: tarefas persistentes

```
const input = document.getElementById('tarefa');
const botao = document.getElementById('adicionar');
const lista = document.getElementById('lista');

let tarefas = JSON.parse(localStorage.getItem('tarefas')) || [];

function salvar() {
  localStorage.setItem('tarefas', JSON.stringify(tarefas));
}

function renderizar() {
  lista.innerHTML = '';

  tarefas.forEach(function (tarefa, indice) {
    const item = document.createElement('li');
    item.textContent = tarefa;

    item.addEventListener('click', function () {
      tarefas.splice(indice, 1);
      salvar();
      renderizar();
    });

    lista.appendChild(item);
  });
}

botao.addEventListener('click', function () {
  const texto = input.value.trim();

  if (texto === '') return;

  tarefas.push(texto);
  input.value = '';
  salvar();
  renderizar();
});

renderizar();
```

Exercícios para fixação

1. Explique bubbling com suas palavras.
2. Crie um exemplo com stopPropagation.
3. Salve seu nome no localStorage e recupere no console.
4. Salve um array de tarefas no localStorage usando JSON.stringify.
5. Explique por que usamos JSON.parse ao recuperar dados.

Gabarito comentado

1. Bubbling é quando o evento começa no elemento alvo e sobe para os pais.

2. Crie uma div com botão; no botão use `event.stopPropagation()`.
3. `localStorage.setItem("nome", "Danrley"); console.log(localStorage.getItem("nome"));`
4. `localStorage.setItem("tarefas", JSON.stringify(tarefas));`
5. Porque o `localStorage` devolve texto; `JSON.parse` converte o texto de volta para array ou objeto.

8. Desenvolvimento web moderno: HTTP, APIs, JSON, Fetch, WebSocket e segurança

Objetivos do capítulo

- Relembrar o modelo de requisições HTTP.
- Entender APIs e troca de dados em JSON.
- Usar `fetch` com `then` e `async/await`.
- Compreender WebSocket para comunicação em tempo real.
- Aplicar validações e práticas básicas de segurança no front-end.

8.1 Requisições HTTP

HTTP é o protocolo usado para comunicação entre navegador e servidor. Quando você acessa uma página ou busca dados de uma API, o navegador faz uma requisição e o servidor envia uma resposta.

Método	Uso comum
GET	Buscar dados
POST	Criar/enviar dados
PUT	Atualizar dados inteiros
PATCH	Atualizar parte dos dados
DELETE	Excluir dados

8.2 API

API é uma ponte de comunicação entre sistemas. No front-end, geralmente chamamos uma API para buscar ou enviar dados.

```
Front-end ---> Requisição HTTP ---> API/Servidor
Front-end <--- Resposta JSON   <--- API/Servidor
```

8.3 JSON

JSON é um formato de dados leve e muito usado em APIs.

```
{
  "id": 1,
  "nome": "Maria",
  "email": "maria@email.com"
}
```

JSON não é objeto JavaScript puro: JSON é texto em formato específico. Para virar objeto JavaScript, precisa ser convertido com `.json()` em `fetch` ou `JSON.parse` quando vem de uma string.

8.4 Fetch com then

```
fetch('https://jsonplaceholder.typicode.com/users')
  .then(function (resposta) {
```

```

    return resposta.json();
  })
  .then(function (usuarios) {
    console.log(usuarios);
  })
  .catch(function (erro) {
    console.error('Erro ao buscar usuários:', erro);
  });

```

8.5 Fetch com async/await

```

async function buscarUsuarios() {
  try {
    const resposta = await fetch('https://jsonplaceholder.typicode.com/users');

    if (!resposta.ok) {
      throw new Error('Erro HTTP: ' + resposta.status);
    }

    const usuarios = await resposta.json();
    console.log(usuarios);
  } catch (erro) {
    console.error('Falha na requisição:', erro);
  }
}

buscarUsuarios();

```

async/await: É uma forma mais legível de trabalhar com operações assíncronas. O await espera a Promise ser resolvida antes de continuar.

8.6 Exibindo dados de API na tela

```

<ul id="usuarios"></ul>

async function carregarUsuarios() {
  const lista = document.getElementById('usuarios');
  lista.textContent = 'Carregando...';

  try {
    const resposta = await fetch('https://jsonplaceholder.typicode.com/users');
    const usuarios = await resposta.json();

    lista.innerHTML = '';

    usuarios.forEach(function (usuario) {
      const item = document.createElement('li');
      item.textContent = usuario.name + ' - ' + usuario.email;
      lista.appendChild(item);
    });
  } catch (erro) {
    lista.textContent = 'Erro ao carregar usuários';
  }
}

carregarUsuarios();

```

8.7 WebSocket

WebSocket mantém uma conexão aberta entre cliente e servidor. É usado quando a aplicação precisa receber dados em tempo real, como chats, jogos, notificações, dashboards e sistemas de atendimento.

```

const socket = new WebSocket('wss://exemplo.com/socket');

socket.addEventListener('open', function () {
  console.log('Conectado');
});

socket.addEventListener('message', function (event) {
  console.log('Mensagem recebida:', event.data);
});

socket.addEventListener('close', function () {

```

```
console.log('Conexão encerrada');
});
```

HTTP comum	WebSocket
Cliente pede e servidor responde	Cliente e servidor ficam conectados
Bom para buscar dados pontuais	Bom para tempo real
Exemplo: carregar lista de usuários	Exemplo: chat online

8.8 Segurança e validação no front-end

Validação no front-end melhora a experiência do usuário, mas não substitui validação no back-end. Usuários mal-intencionados podem alterar código no navegador, então o servidor sempre deve validar de novo.

```
function validarSenha(senha) {
  if (senha.length < 8) {
    return 'A senha deve ter pelo menos 8 caracteres';
  }

  if (!senha.includes('@') && !senha.includes('#')) {
    return 'Use pelo menos um caractere especial';
  }

  return 'Senha válida';
}
```

Nunca confie só no front-end: Tudo que roda no navegador pode ser visto e alterado pelo usuário. Valide dados sensíveis também no servidor.

Exercícios para fixação

1. Explique a diferença entre GET e POST.
2. Crie um fetch que busque usuários e mostre no console.
3. Explique o que é JSON.
4. Diga uma situação em que WebSocket é melhor que HTTP comum.
5. Explique por que validação no front-end não é suficiente.

Gabarito comentado

1. GET busca dados; POST envia/cria dados.
2. Use `fetch(url).then(res => res.json()).then(dados => console.log(dados))`.
3. JSON é um formato textual para representar dados, muito usado em APIs.
4. Chat online, notificações ao vivo ou painel em tempo real.
5. Porque o usuário pode manipular o navegador; o back-end precisa validar novamente.

9. Arrow Functions no JavaScript moderno

Objetivos do capítulo

- Entender a sintaxe das arrow functions.
- Comparar função tradicional e função de seta.
- Usar arrow functions em callbacks e métodos de arrays.
- Compreender cuidados com this, return implícito e objetos retornados.

9.1 Função tradicional x arrow function

```
function somar(a, b) {
  return a + b;
}

const somarArrow = (a, b) => {
  return a + b;
};

const somarCurta = (a, b) => a + b;
```

As três funções acima somam dois números. A arrow function permite escrever funções curtas de forma mais enxuta.

9.2 Regras de sintaxe

Situação	Sintaxe
Sem parâmetro	const ola = () => "Olá";
Um parâmetro	const dobro = n => n * 2;
Dois ou mais parâmetros	const soma = (a, b) => a + b;
Mais de uma linha	Use chaves e return
Retornar objeto direto	Use parênteses: () => ({ nome: "Ana" })

```
const dobro = n => n * 2;

const criarPessoa = nome => ({
  nome: nome,
  ativo: true
});

console.log(dobro(5));
console.log(criarPessoa('Ana'));
```

9.3 Arrow functions em arrays

```
const numeros = [1, 2, 3, 4];

const dobrados = numeros.map(n => n * 2);
const pares = numeros.filter(n => n % 2 === 0);
const soma = numeros.reduce((total, n) => total + n, 0);

console.log(dobrados);
console.log(pares);
console.log(soma);
```

9.4 Arrow functions em eventos

```
const botao = document.getElementById('botao');

botao.addEventListener('click', () => {
  console.log('Clique com arrow function');
});
```

9.5 Cuidado com this

Arrow function não cria seu próprio this. Em muitos callbacks isso é útil, mas em métodos de objetos pode causar confusão.

```
const pessoa = {
  nome: 'Ana',
  falar: function () {
    console.log(this.nome);
  }
};

pessoa.falar(); // Ana
```

```
const pessoa2 = {
  nome: 'Ana',
  falar: () => {
    console.log(this.nome);
  }
};

pessoa2.falar(); // geralmente undefined
```

Regra prática: Use arrow function para callbacks, map, filter, reduce e funções curtas. Evite usar arrow function como método de objeto quando precisar acessar this.

9.6 Erros comuns com arrow function

Erro	Exemplo	Correção
Esquecer return quando usa chaves	<code>n => { n * 2 }</code>	<code>n => { return n * 2 }</code> ou <code>n => n * 2</code>
Retornar objeto sem parênteses	<code>n => { valor: n }</code>	<code>n => ({ valor: n })</code>
Usar this em método de objeto	<code>falar: () => this.nome</code>	<code>falar: function() { return this.nome }</code>

Exercícios para fixação

1. Transforme function dobro(n) { return n * 2; } em arrow function.
2. Use map com arrow function para transformar [1,2,3] em [10,20,30].
3. Use filter com arrow function para pegar idades maiores ou iguais a 18.
4. Explique por que arrow function pode causar problema com this em objetos.

Gabarito comentado

1. `const dobro = n => n * 2;`
2. `const resultado = numeros.map(n => n * 10);`
3. `const maiores = idades.filter(idade => idade >= 18);`
4. Porque arrow function não cria this próprio; ela usa o this do escopo externo.

10. Projetos práticos integradores

A melhor forma de estudar JavaScript para web é construir pequenos projetos. Os projetos abaixo juntam os conteúdos dos capítulos.

10.1 Projeto 1: Lista de tarefas completa

Recursos praticados: DOM, eventos, arrays, criação dinâmica, localStorage, JSON e arrow functions.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Lista de Tarefas</title>
  <style>
    .concluida { text-decoration: line-through; color: gray; }
    .erro { color: red; }
  </style>
</head>
<body>
  <h1>Minhas tarefas</h1>
  <input id="campo" placeholder="Nova tarefa">
  <button id="adicionar">Adicionar</button>
  <p id="mensagem"></p>
```

```

<ul id="lista"></ul>

<script>
  const campo = document.getElementById('campo');
  const adicionar = document.getElementById('adicionar');
  const mensagem = document.getElementById('mensagem');
  const lista = document.getElementById('lista');

  let tarefas = JSON.parse(localStorage.getItem('tarefas')) || [];

  const salvar = () => {
    localStorage.setItem('tarefas', JSON.stringify(tarefas));
  };

  const renderizar = () => {
    lista.innerHTML = '';

    tarefas.forEach((tarefa, indice) => {
      const item = document.createElement('li');
      item.textContent = tarefa.texto;

      if (tarefa.concluida) {
        item.classList.add('concluida');
      }

      item.addEventListener('click', () => {
        tarefas[indice].concluida = !tarefas[indice].concluida;
        salvar();
        renderizar();
      });

      const remover = document.createElement('button');
      remover.textContent = 'Remover';
      remover.addEventListener('click', event => {
        event.stopPropagation();
        tarefas.splice(indice, 1);
        salvar();
        renderizar();
      });

      item.appendChild(remover);
      lista.appendChild(item);
    });
  };

  adicionar.addEventListener('click', () => {
    const texto = campo.value.trim();

    if (texto === '') {
      mensagem.textContent = 'Digite uma tarefa';
      mensagem.classList.add('erro');
      return;
    }

    tarefas.push({ texto: texto, concluida: false });
    campo.value = '';
    mensagem.textContent = '';
    salvar();
    renderizar();
  });

  renderizar();
</script>
</body>
</html>

```

10.2 Projeto 2: Catálogo de produtos com filtro

Recursos praticados: arrays de objetos, filter, map, DOM e eventos de input.

```

const produtos = [
  { nome: 'Notebook', preco: 3000 },
  { nome: 'Mouse', preco: 50 },
  { nome: 'Teclado', preco: 120 },
  { nome: 'Monitor', preco: 900 }
];

```

```
const busca = document.getElementById('busca');
const lista = document.getElementById('produtos');

const renderizar = itens => {
  lista.innerHTML = '';

  itens.forEach(produto => {
    const li = document.createElement('li');
    li.textContent = `${produto.nome} - R$ ${produto.preco}`;
    lista.appendChild(li);
  });
};

busca.addEventListener('input', () => {
  const termo = busca.value.toLowerCase();

  const filtrados = produtos.filter(produto =>
    produto.nome.toLowerCase().includes(termo)
  );

  renderizar(filtrados);
});

renderizar(produtos);
```

10.3 Projeto 3: Consulta de usuários por API

Recursos praticados: fetch, async/await, tratamento de erro, DOM e renderização dinâmica.

```
const lista = document.getElementById('usuarios');
const carregar = document.getElementById('carregar');

carregar.addEventListener('click', async () => {
  lista.textContent = 'Carregando...';

  try {
    const resposta = await fetch('https://jsonplaceholder.typicode.com/users');

    if (!resposta.ok) {
      throw new Error('Erro ao buscar dados');
    }

    const usuarios = await resposta.json();
    lista.innerHTML = '';

    usuarios.forEach(usuario => {
      const item = document.createElement('li');
      item.textContent = `${usuario.name} - ${usuario.email}`;
      lista.appendChild(item);
    });
  } catch (erro) {
    lista.textContent = 'Não foi possível carregar usuários.';
  }
});
```

10.4 Checklist para entregar um projeto

- O HTML está organizado e sem tags abertas?
- O CSS está separado ou bem organizado?
- O JavaScript está em arquivo separado quando possível?
- Os nomes de variáveis são claros?
- Os eventos usam addEventListener?
- Campos obrigatórios são validados?
- Há mensagens claras para erro e sucesso?
- O código trata lista vazia?
- O localStorage usa JSON.stringify e JSON.parse corretamente?
- O console do navegador está sem erros?

11. Plano de estudo e revisão rápida

Este plano foi montado para estudar de forma progressiva. A ideia é revisar teoria, copiar exemplos e fazer exercícios.

Dia	Conteúdo	Prática recomendada
1	HTML, CSS e papel do JavaScript	Criar página simples com script externo
2	Variáveis, tipos e operadores	Criar cálculos e mensagens no console
3	Condicionais e laços	Exercícios de nota, idade e repetição
4	Funções	Criar funções de cálculo e validação
5	DOM: seleção e alteração de texto	Alterar títulos, parágrafos e estilos
6	Inputs e formulários	Ler campos e validar vazio
7	Criação dinâmica de elementos	Criar lista de tarefas básica
8	Eventos	click, input, submit e preventDefault
9	Arrays	push, splice, forEach, map, filter
10	Eventos avançados	Delegação e stopPropagation
11	localStorage e JSON	Salvar e recuperar lista de tarefas
12	Fetch e APIs	Buscar dados e renderizar na tela
13	Arrow functions	Refatorar códigos anteriores
14	Projeto final	Montar app completo e revisar erros

11.1 Revisão rápida por tema

Tema	Você precisa saber responder
Variáveis	Quando usar let e const?
Condicionais	Como o if decide o fluxo?
Laços	Como repetir uma tarefa sem duplicar código?
DOM	Como selecionar e alterar elementos?
Eventos	Como reagir ao clique e ao submit?
Arrays	Como guardar e percorrer vários dados?
Persistência	Como salvar dados no navegador?
Fetch	Como buscar dados de uma API?
Arrow functions	Como simplificar callbacks?

11.2 Roteiro para resolver qualquer exercício

1. Leia o enunciado e identifique entradas, processamento e saída.
2. Liste quais elementos HTML você precisa.
3. Dê id para os elementos que serão acessados pelo JavaScript.
4. Selecione os elementos no script.
5. Crie as variáveis de controle.
6. Crie as funções principais.

7. Adicione eventos com `addEventListener`.
8. Teste no navegador e observe o console.
9. Corrija erros um por vez.
10. Refatore o código para ficar mais claro.

12. Exercícios finais com gabarito

Resolva antes de olhar o gabarito. O objetivo é treinar raciocínio e não apenas copiar código.

1. **Fundamentos:** Crie uma função que receba uma nota e retorne aprovado se for maior ou igual a 6.
2. **DOM:** Crie um botão que altere o texto de um `h1`.
3. **Formulário:** Crie um input de nome e mostre uma saudação ao clicar em enviar.
4. **Array:** Dado `[10, 20, 30]`, use `reduce` para calcular a soma.
5. **map:** Transforme `[1, 2, 3]` em `[2, 4, 6]`.
6. **filter:** Filtre apenas produtos com preço maior que 100.
7. **createElement:** Crie um `li` pelo JavaScript e adicione a uma `ul`.
8. **Evento:** Use `preventDefault` em um formulário e mostre mensagem de sucesso.
9. **localStorage:** Salve um objeto usuário no `localStorage`.
10. **fetch:** Busque dados de uma API e trate erro com `try/catch`.
11. **Arrow function:** Transforme uma função tradicional em arrow function.
12. **Projeto:** Crie uma lista de tarefas com adicionar, remover e salvar no `localStorage`.

12.1 Gabarito comentado dos exercícios finais

Fundamentos

```
function verificarNota(nota) {
  return nota >= 6 ? 'Aprovado' : 'Reprovado';
}
```

DOM

```
const titulo = document.getElementById('titulo');
const botao = document.getElementById('botao');

botao.addEventListener('click', () => {
  titulo.textContent = 'Texto alterado';
});
```

Formulário

```
const nome = document.getElementById('nome');
const enviar = document.getElementById('enviar');
const resultado = document.getElementById('resultado');

enviar.addEventListener('click', () => {
  resultado.textContent = `Olá, ${nome.value}`;
});
```

Array reduce

```
const numeros = [10, 20, 30];
const soma = numeros.reduce((total, n) => total + n, 0);
```

map

```
const dobrados = [1, 2, 3].map(n => n * 2);
```

filter

```
const caros = produtos.filter(produto => produto.preco > 100);
```

createElement

```
const ul = document.getElementById('lista');
const li = document.createElement('li');
li.textContent = 'Novo item';
ul.appendChild(li);
```

Evento submit

```
formulario.addEventListener('submit', event => {
  event.preventDefault();
  mensagem.textContent = 'Enviado com sucesso';
});
```

localStorage

```
const usuario = { nome: 'Danrley', curso: 'JavaScript' };
localStorage.setItem('usuario', JSON.stringify(usuario));
```

fetch

```
async function buscar() {
  try {
    const resposta = await fetch('https://jsonplaceholder.typicode.com/users');
    const dados = await resposta.json();
    console.log(dados);
  } catch (erro) {
    console.error('Erro:', erro);
  }
}
```

Arrow function

```
// Tradicional
function somar(a, b) { return a + b; }

// Arrow
const somar = (a, b) => a + b;
```

Projeto

Use o projeto completo do capítulo 10 como base. Ele possui array de tarefas, renderização no DOM, remoção por índice e persistência no localStorage.

13. Glossário e referências dos materiais enviados

Termo	Significado
API	Interface usada para comunicação entre sistemas.
Array	Lista ordenada de valores em JavaScript.
Callback	Função passada como argumento para ser executada depois.
DOM	Representação da página HTML em objetos manipuláveis pelo JavaScript.
Evento	Ação ou acontecimento detectado pelo navegador.
Fetch	API do navegador usada para fazer requisições HTTP.

JSON	Formato textual usado para representar dados.
localStorage	Armazenamento persistente no navegador.
Node/Nó	Cada elemento ou parte da árvore DOM.
Promise	Objeto que representa uma operação assíncrona.
querySelector	Método que seleciona elemento por seletor CSS.
WebSocket	Comunicação persistente e bidirecional em tempo real.

13.1 Materiais usados como base

- 1 - Introdução à Programação Web com JavaScript.pdf
- 1_1_Javascript na Web e Introdução ao DOM.pdf
- 2 - Manipulando o DOM.pdf
- 3 - Construção Dinâmica de Interfaces.pdf
- 4 - Eventos Web e Experiência do Usuário.pdf
- 5 - Listas em JavaScript e a Web.pdf
- 6 - Manipulação Avançada de Eventos e Persistência de Dados.pdf
- 7 - Desenvolvimento Web Moderno.pdf
- 8 - Arrow Functions no JavaScript Moderno.pdf

13.2 Checklist final para prova

- Sei explicar a diferença entre HTML, CSS e JavaScript.
- Sei criar variáveis com let e const.
- Sei usar if, else, for e funções.
- Sei selecionar elementos com getElementById e querySelector.
- Sei alterar textContent, innerHTML, style e classList.
- Sei criar elementos com createElement e inserir com appendChild.
- Sei usar addEventListener para click, input e submit.
- Sei manipular arrays com push, splice, forEach, map, filter e reduce.
- Sei salvar e recuperar dados com localStorage e JSON.
- Sei buscar dados com fetch e async/await.
- Sei escrever arrow functions e reconhecer cuidados com this.